

CODES

QUESTION 2:

Candidate Elimination Algorithm for Crop Disease Prediction

```
import pandas as pd
```

Step 1: Dataset

```
data = [
    ['High', 'High', 'Heavy', 'Wet', 'Yes'],
    ['High', 'High', 'Light', 'Wet', 'Yes'],
    ['Medium', 'High', 'Heavy', 'Wet', 'Yes'],
    ['Low', 'Normal', 'Light', 'Dry', 'No'],
    ['Medium', 'Normal', 'Light', 'Dry', 'No']
]

columns = ['Temp', 'Humidity', 'Rainfall', 'Soil', 'Risk']
df = pd.DataFrame(data, columns=columns)
```

Step 2: Initialize S and G

```
S = ['∅'] * (len(columns) - 1)
G = [['?'] * (len(columns) - 1)]
```

Function to check consistency

```
def is_consistent(h, x):
    return all(h[i] == x[i] or h[i] == '?' for i in range(len(h)))

print("Initial S:", S)
print("Initial G:", G)
print("\n")
```

Step 3: Training

```
for index, row in df.iterrows():
    x = row[:-1].tolist()
    y = row[-1]

    print(f"Processing Example {index+1}: {x} -> {y}")

    if y == 'Yes': # Positive Example
        # Remove inconsistent hypotheses from G
        G = [g for g in G if is_consistent(g, x)]

        # Generalize S
        for i in range(len(S)):
            if S[i] == '∅':
                S[i] = x[i]
            elif S[i] != x[i]:
                S[i] = '?'
    else: # Negative Example
        new_G = []
        for g in G:
            if is_consistent(g, x):
                for i in range(len(g)):
                    if g[i] == '?':
```

```

        if S[i] != x[i]:
            new_h = g.copy()
            new_h[i] = S[i]
            new_G.append(new_h)
        else:
            new_G.append(g)
    G = new_G

    print("S:", S)
    print("G:", G)
    print("-----")

# Final Version Space
print("\nFinal Specific Hypothesis (S):", S)
print("Final General Hypotheses (G):", G)

```

QUESTION 3 :

```

from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.preprocessing import LabelEncoder
import matplotlib.pyplot as plt

# Define features (X) and target (y)
X = df.drop('Risk', axis=1)
y = df['Risk']

# Convert categorical features in X to numerical using one-hot encoding
X = pd.get_dummies(X, drop_first=True)

# Convert categorical target y to numerical using LabelEncoder
le = LabelEncoder()
y = le.fit_transform(y)

model = DecisionTreeClassifier(criterion='entropy')
model.fit(X, y)

plt.figure(figsize=(10,6))
plot_tree(model, feature_names=X.columns, class_names=le.classes_,
filled=True)
plt.show()

```

QUESTION 4 :

```

from sklearn.naive_bayes import GaussianNB
model = GaussianNB()
model.fit(X, y)
pred = model.predict(X)
print(pred)

```

QUESTION 5 :

```

from sklearn.neighbors import KNeighborsClassifier
for k in range(1,6):
    model = KNeighborsClassifier(n_neighbors=k)

```

```
model.fit(X, y)
print(f"K={k}, Accuracy={model.score(X,y)}")
```

QUESTION 6 :

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X, data['Yield'])
pred = model.predict(X)
print(pred)
```

QUESTION 7:

```
from sklearn.neural_network import MLPClassifier

model = MLPClassifier(hidden_layer_sizes=(5,5), max_iter=1000)
model.fit(X, y)

print("Accuracy:", model.score(X,y))
```